

Git ve GitHub

Projenin geçmişini tut, takımınla çalış, emeğini görünür kıl.

Arif Solmaz
İSTÜN Mekatronik Mühendisliği



45 dakikalık rota

Bir robot projesi, dört durak.

1. Temeller: dosya, depo ve geçmiş · 9 dk 15 sn
2. Değişiklik: bilgisayardan GitHub'a · 12 dk 45 sn
3. Takım: issue, dal ve inceleme · 12 dk
4. Paylaşım, kaynaklar ve sorular · 11 dk

32 ana slayt · 12 başvuru slaytı · Tek proje üzerinden anlatım

Son dosya hangisi?

robot_final_SON.ino dosyası gerçekten son sürüm mü?

SORUN

Hangisi son hali?

23:47

Robot Projesi


robot.ino


geçmiş: her değişiklik, kim, ne zaman

Robot Takımı

Kaan

Elif, son kodu atar mısın?

Elif

robot_final.ino'yu attım

robot_final.ino

Deniz

Ama ben sensörü robot_v2'de düzeltmiştim...

Kaan

Robot yine çizgiden çıktı. Hangisi son hali??

Elif

Bilmiyorum. Hepsini tek tek mi deneyelim?

GitHub'da tek bir **robot.ino** var; eski hâlleri geçmişinde saklı.

Dosya adlarına sürüm yazmak yerine, değişikliklerin geçmişini tutarız.

Git ve GitHub

Git geçmişi tutar. GitHub bu geçmiş etrafında birlikte çalışmayı sağlar.

Git • bilgisayarındaki araç

Commit'leri, dalları ve yerel geçmişi yönetir. Birçok işlem internet olmadan çalışır.

GitHub • ortak platform

Depoyu barındırır; issue, pull request ve inceleme araçlarını bir araya getirir.

Git aynı zamanda GitLab ve diğer Git sunucularıyla da kullanılabilir.

01 · TEMELLER

Depo dosya ve geçmiştir

Kod, devre şeması, rapor ve açıklama aynı projenin parçalarıdır.

1 DAKİKA

Projeni GitHub'a koy

E Elif'in ekranı

github.com/elif/robot-projem

elif / robot-projem Public

<> Code Issues 0 Pull requests 0 Actions

main Add file <> Code

E elif Proje dosyaları eklendi 2 Commits

fotograflar Proje dosyaları eklendi

README.md Initial commit

devre-semasi.png Proje dosyaları eklendi

proje-raporu.pdf Proje dosyaları eklendi

robot.ino Proje dosyaları eklendi

README

robot-projem

Çizgi izleyen robot — ders projesi

SENİN İÇİN

Sadece kod değil: projenin her şeyi

✓ Arduino kodu

✓ Devre şeması

✓ Proje raporu

✓ Fotoğraflar

Repository (depo) = projenin internetteki klasörü.

Bitti: kod, devre şeması, rapor ve fotoğraflar artık internette.

Repository = depo · robot.ino + README.md + devre şeması + kayıtlı geçmiş

Commit bir kayıt noktası

“IR sensör eşiği artırıldı” bir değişikliği ve amacını anlatır.

- Projenin seçilmiş hâlini geçmişe kaydeder.
- Bir kimliği, yazarı ve açıklama mesajı vardır.
- Bilgisayarda commit etmek, GitHub'a göndermekten ayrıdır.

Kaydetmek ≠ commit etmek ≠ push etmek

README giriş kapısıdır

Projeyi açan biri neye baktığını birkaç cümlede anlayabilmeli.

```
# Robot Projem
```

```
IR sensörlerle çizgi izleyen Arduino robotu.
```

```
## Donanım
```

```
Arduino, IR sensörler, motor sürücü ve motorlar.
```

```
## Çalıştırma
```

```
Bağlantı şemasını ve robot.ino açıklamalarını izleyin.
```

Amaç • donanım • kurulum • fotoğraf veya şema

Kim görebilir?

Görünürlük ve yazma izni ayrı kararlardır.

Public • herkese açık

Herkes görebilir ve klonlayabilir. Ana depoya yazmak ayrıca izin gerektirir.

Private • özel

Erişimi yetkili kişilerle sınırlıdır. Takım üyelerine uygun izin verilir.

Parolalar, erişim anahtarları ve kişisel veriler proje dosyası değildir.

02 · DEĞİŞİKLİĞİN YOLU

Dün çalışıyordu

Geçmiş, “Ne değişti?” sorusuna somut bir cevap verir.

2 FAYDA

Zaman makinesi: her değişiklik kayıtlı

E Elif'in ekranı

github.com/elif/robot-projem/commit/7f3e2a1

elif / robot-projem Public

<> Code Issues 0 Pull requests 0 Actions

Sensör eşiği değiştirildi

deniz committed yesterday 7f3e2a1

<> Browse files

1 file changed

robot.ino +1 -1

3 int VIRAJ_HIZI = 120; // virajda hız

4 - int ESİK = 500; // sensör eşiği

4 + int ESİK = 900; // sensör eşiği

5

6 void setup() {

SENİN İÇİN

Proje için zaman makinesi

✓ Hiçbir değişiklik kaybolmaz.

✓ Kim, ne zaman, neyi değiştirdi: hepsi kayıtlı.

✓ Bozmaktan korkma: her kayıt bir geri dönüş noktası.

Commit = bir kayıt anı, yanında kısa bir not.

Buldun: eşik 500'den 900'e çıkmış. O günkü hâline dönüp bakabilirsin.

Kim, ne zaman, hangi kayıtlı? · Geçmiş yalnız commit edilen değişiklikleri kapsar.

Satır farkı nasıl okunur?

Diff, iki hâl arasındaki değişikliği gösterir.

```
robot.ino
```

```
- const int ESIK = 500;  
+ const int ESIK = 900;
```

– önceki satır · + yeni satır · ESIK: IR sensör okuması için örnek eşik

Değişiklik dört yerde durur

Düzenle → hazırla → kaydet → paylaş



Add hazırlar. Commit yerelde kaydeder. Push gönderir. Pull uzaktaki yenilikleri alır.

Clone ile yerel kopya

Var olan depoyu dosyaları ve geçmişiyle bilgisayara alırız.

```
git clone <depo-adresi>  
cd robot-projem
```

Public depo HTTPS ile okunabilir; private depoda erişim gerekir. ZIP indirmek Git geçmişini getirmez.

Önce dosyayı değiştiririz

README'ye çizgi sensörlerinin görevini ekledik ve dosyayı kaydettik.

```
README.md
```

```
## Sensörler
```

```
IR sensörler çizgiyi algılamak için kullanılır.
```

```
Bağlantılar devre-semasi.png dosyasındadır.
```

```
# Dosyayı kaydet → ardından:
```

```
git status
```

Düzenleme adımı olmadan add ve commit yeni bir değişiklik üretmez.

Add ile değişikliği seçeriz

Bir commit'in kapsamını bilinçli belirleriz.

```
git add README.md  
git diff --cached
```

Add, dosyanın o andaki hâlini hazırlar. Sonraki düzenleme için yeniden add gerekebilir.

Commit ile yerelde kaydederiz

Mesaj, değişikliğin ne işe yaradığını anlatır.

```
git commit -m "IR sensör bağlantıları README'de açıklandı"  
git log --oneline
```

Bu kayıt bilgisayardadır. GitHub'a aktarım bir sonraki adımdır.

Push ile GitHub'a göndeririz

Yerel kayıtlar, push sonrasında uzak depoda görünür.

```
git push
```

Yazma izni ve kimlik doğrulama gerekir. GitHub hesap parolası, Git için HTTPS parolası değildir.

Pull ile yenilikleri alırız

Takımın ortak geçmişi ile yerel kopya yeniden buluşur.

```
git status  
git pull --ff-only
```

Örnek, yerel dalda ayrı commit yoksa ilerler. Geçmişler ayrılmışsa birleştirme yöntemi seçilir.

Issue işi görünür kılar

“IR sensör eşiğini incele” ortak bir iş kaydıdır.

3 FAYDA Takımın yapılacaklar listesi E Elif'in ekranı

github.com/elif/robot-projem/issues

elif / robot-projem Public

<> Code Issues 2 Pull requests 0 Actions

Q is:issue state:open Labels New issue

2 Open ✓ 1 Closed Author ▾ Labels ▾ Assignee ▾

- ✓ Rapor için robotun fotoğrafını çek rapor #3 opened 2 minutes ago by elif
- ✓ Motor sürücüsü ısınıyor bug #1 opened 2 days ago by kaan

SENİN İÇİN

Kim neyi yapıyor, herkes görür

- ✓ İşler mesaj grubunda kaybolmaz; aylar sonra bile bulunur.
- ✓ Her işin bir sorumlusu var.
- ✓ Bitirme projesi, kulüp, ödev grubu: hepsine uyar.

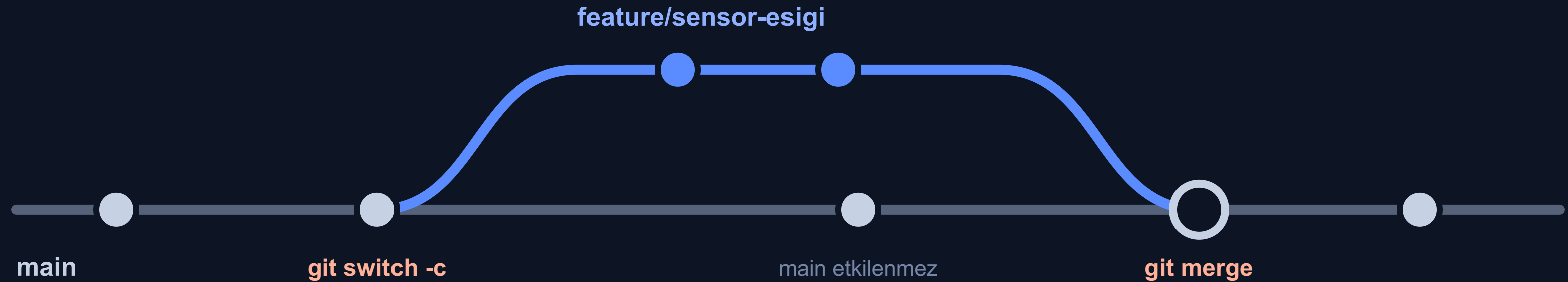
Issue = yapılacak bir iş ya da bir hata notu.

İş bitince **Close issue** : listeden düşer, kaydı kalır.

Başlık + gözlenen durum + beklenen sonuç + sorumlu

Dal ayrı bir çalışma çizgisidir

Aynı başlangıçtan farklı değişiklikler ilerleyebilir.



Branch, bütün dosyaların fiziksel kopyası değil, bir commit'e işaret eden addır.

Başlangıç için sade dal düzeni

Dal düzenini projenin ihtiyacı belirler.

Ana çizgi

main • takımın kabul ettiği güncel çalışma.

İş dalları

feature/sensor-esigi • belirli değişiklik, inceleme, birleştirme.

dev veya staging her proje için zorunlu değildir. Hedef dal CONTRIBUTING kılavuzundan öğrenilir.

Pull request bir öneridir

“Bu dalın değişikliğini main'e alalım mı?”

Kaynak: `feature/sensor-esigi` → Hedef: `main`

Başlık: IR sensör eşiğini artır

Neden: Robot bazı koşullarda çizgiyi kaybediyordu.

Değişiklik: `ESIK`, 500 → 900.

Doğrulama: Yapılan gözlem ve test burada belirtilir.

PR açıklaması neyin, neden değiştiğini ve nasıl doğrulandığını anlatır.

İnceleme çözümü güçlendirir

Bir sayı değişikliği küçük görünür; robotun davranışını etkileyebilir.

- Files changed: değişen satırlar ve bağlam.
- Yorum: “Bu eşik hangi koşulda denendi?”
- Yeni commit aynı dala gönderilince PR güncellenir.
- Onay ve gerekli kontroller sonrası birleştirme.

İnceleme, kararın ve gerekçesinin projede kalmasını sağlar.

Birleştirme sonrası herkes günceller

GitHub'da kabul edilen değişiklik, yerel main'e ayrıca alınır.

```
git switch main  
git pull --ff-only
```

1. PR, main'e birleştirilir.
2. Yerelde main dalına geçilir.
3. Uzak değişiklikler alınır.
4. Birleşmiş iş dalı gerekirse temizlenir.

Dal silmeden önce değişikliğin hedef dalda bulunduğu doğrulanır.

Çakışma kararı bize bırakır

İki dal aynı satırı farklı biçimde değiştirmiş.

```
robot.ino
```

```
<<<<<< HEAD
```

```
const int ESIK = 900;
```

```
=====
```

```
const int ESIK = 700;
```

```
>>>>>> main
```

HEAD: şu anki dal • main: birleştirdiğimiz dal • ESIK: IR sensör okuma eşiği

Çakışmayı anlamıyla çözeriz

Önce doğru davranışa karar verilir, ardından işaretler temizlenir.

```
git add robot.ino  
git commit -m "IR sensör eşik çakışması  
çözüldü"
```

1. İki değişikliğin gerekçesini ve bağlamını oku.
2. Proje için uygun çözümü seç veya yeni çözüm yaz.
3. İşaretleri kaldır, dosyayı kaydet ve doğrula.
4. Dosyayı add ile hazırla, merge commit'ini tamamla.

Örnek bir merge çözümüdür; devam komutu yapılan işleme göre değişir.

Hazır kodu nasıl seçeriz?

Robota HC-SR04 mesafe sensörü eklerken ilk sonuç tek başına yeterli değildir.

- Donanım ve kullanılan yazılım sürümüyle uyum.
- README, bağlantı açıklaması ve çalışan örnekler.
- Bakım durumu, açık sorunlar ve yayınlar.
- Lisans ve projenin kullanım koşulları.

Yıldız, ilgi ve kaydetme işaretidir; kalite veya kullanım garantisi değildir.

Lisans kullanım şartlarını anlatır

Bir kodu görebilmek, onu her amaçla kullanabilmek anlamına gelmez.

- LICENSE dosyasını ve bağımlılıkların lisanslarını oku.
- Kopyalama, dağıtma ve değişiklik koşullarını öğren.
- Gerekli atıf ve lisans bildirimlerini koru.

Açık kaynak kullanırken teknik uygunluk ve kullanım izni birlikte değerlendirilir.

Fork ile dış projeye katkı

Kendi kopyanda çalış, asıl projeye değişiklik öner.

1. Katkı kılavuzunu oku ve hedef dalı öğren.
2. Depoyu hesabına fork et; kendi fork'unu klonla.
3. İş dalında değiştir, commit et ve fork'una gönder.
4. Asıl projeye doğru hedef dalı seçerek PR aç.

Public depoyu doğrudan klonlamak mümkündür; fork, kendi uzak kopyanda çalışmanı sağlar.

Portfolyo projeyi anlaşılır kılar

İyi açıklanan birkaç proje, katkı sayısından daha çok şey anlatabilir.

Projede göster

Amaç, senin katkın, kullanılan donanım, sonuçlar ve çalışma sınırları.

Profilde düzenle

Öne çıkan depolar, açık README'ler ve doğru çalışan proje bağlantıları.

Katkı grafiği GitHub'ın saydığı etkinlikleri gösterir; bütün çalışma günlerini veya emeği ölçmez.

Öğrenci kaynakları

GitHub Education, doğrulanmış öğrenciler için araç ve öğrenme imkânları sunar.

- Student Developer Pack: değişen iş ortağı teklifleri.
- Copilot Student: doğrulanmış öğrenciler için ücretsiz erişim.
- Başvuru: öğrenci durumu ve gerekli belgelerle doğrulama.

Güncel şartlar ve teklifler: education.github.com/pack

ÖZET

Bir projenin bütün akışı

Sorun → issue → dal → değişiklik → commit → push → PR → inceleme → merge → pull

- Git kaydeder; GitHub ortak çalışmayı düzenler.
- Kaydet, commit ve push farklı adımlardır.
- README projeyi, issue işi, PR çözümü açıklar.

Kaynaklar: GitHub Docs · Pro Git · GitHub Skills

KAPANIŞ

Sorular

Değişiklik nerede? Kim görebiliyor? Takıma nasıl ulaşıyor?

Git · GitHub · repository · commit · branch · issue · pull request

[GitHub Docs](#)

[Pro Git · Türkçe](#)

[GitHub Skills](#)

[GitHub Education](#)

Git kurulumu

İşletim sistemine uygun resmî kurulum yolunu seç.

```
git --version
```

Windows

Git for Windows; Git Bash veya uygun terminal.

macOS ve Linux

İşletim sistemine uygun paket veya kurulum yöntemi.

Sürüm numarası sisteme göre değişir. Güncel kurulum: git-scm.com/install

Commit kimliği

Kayıtlarda görünen ad ve e-posta önceden ayarlanır.

```
git config --global user.name "Elif Örnek"  
git config --global user.email "elif@example.com"  
git config --global init.defaultBranch main
```

Ad ve e-posta örnektir. Bu ayar GitHub'da kimlik doğrulama yapmaz.

Editör ve terminal

Aynı Git işlemi terminalde veya grafik arayüzde yapılabilir.

Terminal

Komut ve seçenekleri açıkça görürsün; işlem adımları kolayca paylaşılır.

VS Code

Dosyalar, fark görünümü, Source Control ve entegre terminal aynı pencerededir.

Open Folder ile proje açılır; View → Terminal ile terminal görünür.

.git ve .gitignore

Geçmişin veritabanı ile izleme tercihleri farklı dosyalarda durur.

```
robot-projem/  
  .git/  
  .gitignore  
  README.md  
  robot.ino  
  
# .gitignore örneği  
build/  
.env
```

.git elle düzenlenmez. .gitignore, zaten izlenen dosyaları geçmişten silmez.

SSH anahtar çifti

GitHub'a açık anahtar eklenir; özel anahtar kişide kalır.

```
ssh-keygen -t ed25519 -C "elif@example.com"
```

```
# GitHub'a eklenecek dosya:
```

```
~/.ssh/id_ed25519.pub
```

```
# Paylaşılmayacak özel anahtar:
```

```
~/.ssh/id_ed25519
```

Anahtar için parola kullanımı koruma sağlar. Mevcut anahtarın üzerine yazılmaz.

SSH ajanı ve bağlantı

Ajanın kurulumu işletim sistemine göre değişir.

```
# Bash / Git Bash örneği  
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

```
# GitHub bağlantısını kontrol et  
ssh -T git@github.com
```

macOS Keychain ve Windows OpenSSH adımları farklı olabilir; resmî yönerge izlenir.

Yerel klasörden depoya

Bu örnek, GitHub'da boş bir uzak depo varsayar.

```
cd robot-projem  
git init -b main  
git add README.md robot.ino  
git commit -m "Robot projesinin ilk kaydı"  
git remote add origin <depo-adresi>  
git remote -v
```

README.md ve robot.ino önceden vardır. Uzak depoda başlangıç commit'i varsa clone daha sade olabilir.

İlk gönderim ve izleme

-u, yerel dalın hangi uzak dalı izleyeceğini kaydeder.

```
git push -u origin main
```

```
# İzleme bağlantısı kurulduktan sonra:
```

```
git push
```

```
git pull --ff-only
```

Dal adını git branch ile kontrol et. Komutta projenin gerçek dal adı kullanılır.

Dal ve fark komutları

Karşılaştırmada başlangıç ve sonuç tarafı açık yazılır.

```
git branch
git switch -c feature/sensor-esigi

# İki dalın son hâli: main → feature
git diff main feature/sensor-esigi -- robot.ino

# Ortak başlangıçtan feature dalındaki değişiklik:
git diff main...feature/sensor-esigi
```

Tab tamamlama kabuk ayarına bağlıdır; yazılan örnek dal adının başlangıcı olmalıdır.

Geri alırken niyeti seç

Dosya, hazırlık alanı ve geçmiş için farklı işlemler vardır.

Hazırlıktan çıkar

`git restore --staged README.md` Çalışma dosyasındaki değişiklik korunur.

Paylaşılan değişikliği tersine çevir

`git revert <commit>` Önceki etkiyi geri alan yeni commit oluşturur.

`git reset HEAD~1`, dalı önceki commit'e taşır; varsayılan mixed çalışma dosyalarını korur.

Hard reset ve kurtarma sınırları

Commit edilmemiş çalışma, hard reset ile kaybolabilir.

```
# Yalnız etkileri değerlendirilmişse:  
git reset --hard <commit>  
  
# Yerel işaretçi hareketlerini incele:  
git reflog
```

Reflog bazı eski commit'leri bulmaya yardımcı olur; kaydedilmemiş bütün işi kurtarma garantisi vermez.

Komutlara kısa bakış

Komutlar, ana dersteki kavramların kısa karşılığıdır.

```
git clone <depo-adresi>
# Dosyayı düzenle ve kaydet
git status
git add <dosya>
git commit -m "Ne ve neden değişti?"
git push

# Takımın yeniliklerini al
git pull --ff-only
# İş dalı oluştur
git switch -c feature/<is>
```

Yer tutucular gerçek değerlerle değiştirilir. Kurulum, kimlik ve izinler önce sağlanır.